

C 语言程序设计

高频考点与易错点贯通笔记

基于历年考试材料整理

核心原则 先弄清类型和地址 再跟踪程序执行 最后独立写完大题

一 试卷重点与使用方法

这份笔记不按教材章节机械重排，而是围绕历年试卷中反复出现的知识关系展开。目标是让读者知道每类题实际在检查什么、应当怎样判断，以及完整程序最容易在哪些环节失分。

最本质的复习原则：题型与重点较固定，必须通过重复练习把知识转化为稳定步骤。大题在考前一定要脱离答案完整写出；看懂答案不等于会写，看过答案后仍要重新独立完成一次。

1.1 稳定题型与分值

题型	常见分值	真正考查的能力	主要失分来源
判断	10	识别语法、类型和存储是否合法	只看形式，不检查变量是否声明、类型是否匹配
表达式	10	按类型和运算规则得到确定结果	忽略整数除法、无符号转换、短路和副作用
程序改错	10	从编译、类型、逻辑和边界四层定位问题	只改表面语法，未恢复正确的数据流
程序填空	10	从上下文反推循环不变量和函数接口	局部空格能填，整体流程不闭合
输出结果	25	逐句跟踪数组、指针、静态变量和结构体	凭感觉跳步，未记录变量变化
编写程序	35	把需求拆成函数并写成完整可运行程序	函数接口、边界条件和字符串结束符不完整



华中科技大学 2025~2026 学年第一学期 “C 语言程序设计”考试试卷 (A 卷)

考试方式 闭卷 考试时间 2026 年 01 月 20 日下午 考试时长 150 分钟

题号	一	二	三	四	五	六	总分	总分人	核对人
分值	10	10	10	10	25	35	100		
得分									

图 1 最新试卷仍保持六类固定题型 编程和读程序合计占六成

1.2 知识优先级

等级	知识模块	覆盖题型
A	数组 指针 字符串 函数参数	判断 改错 填空 输出 编程
A	表达式类型转换 短路 自增 位运算	判断 表达式 输出
A	循环 分支 排序 查找 字符处理	改错 填空 输出 编程
A	结构体数组 结构体指针 综合数据处理	判断 输出 编程
B	作用域 生存期 static 全局变量	判断 输出
B	递归 动态内存 标准库函数	旧题中出现 需掌握基本写法

阅读顺序建议：先完成第二章和第四章，再做第五章。第三章解决读程序的执行过程，第六章负责把前面的知识组合成完整大题。

二 数据类型 定义与表达式

判断题和表达式题的共同起点只有两个：这个写法是否合法，以及每个子表达式的类型是什么。先确定类型，再计算数值。

2.1 变量 常量与合法定义

标识符：由字母、数字和下划线组成，首字符不能是数字，不能使用关键字。大小写不同代表不同标识符。

声明与赋值：声明只对该声明语句中出现的变量生效。若 b、c 尚未声明，`int a=b=c=5;` 不合法。正确写法是 `int a=5, b=5, c=5;`，或先声明再执行 `a=b=c=5;`。

科学计数法：e 或 E 后必须是整数指数，例如 `1.45e+3`。指数部分不能写成小数。

进制常量：前缀 0 表示八进制，数字只能为 0 至 7；前缀 0x 或 0X 表示十六进制。

字符与转义：考试通常按 ASCII 解释字符值。八进制转义最多读取三个八进制数字；十六进制转义从 x 后持续读取十六进制数字。

```
int count = 10;

unsigned int mask = 0x55AAu;

char newline = '\n';

char letter = 'A';

char text[] = "HUST"; /* 实际占 5 字节，末尾还有 '\0' */
```

2.2 数组定义中的类型检查

- 一维数组元素个数必须足以容纳初始化数据；未显式初始化的剩余元素自动置零。
- 二维数组初始化时可以省略第一维，但不能省略后续维度，因为编译器必须知道一行占多少字节。
- 数组名在多数表达式中转换为首元素指针，但数组本身不是可以重新赋值的指针变量。
- 字符串数组的容量必须包含结尾的空字符。`char s[4]="2026"` 没有空间存放完整字符串。

成绩	分数								

分 数	
评卷人	

一、判断下列语句或程序段的对错。（“×”表示错，“√”表示对）（10 分）

(1) char x=0, y=x; ()

(2) int xyz-2=1; ()

(3) printf("%d\n", (40,50,60)); ()

(4) float x[10];
int *p = x; ()

(5) register int x;
scanf("%d",&x); ()

(6) char str[20];
str="hust aia"; ()

(7) int *p, a[2][3];
p = *a+1; ()

(8) int s[] = {5,6,7,8}; ()

(9) char **pp, a[4][5];
pp = a; ()

(10) int x='\107'; ()

分 数	
评卷人	

二、计算下列表达式的值（10 分）

假设 int 和 unsigned int 类型均为 16 位长度，且各题彼此独立
设 unsigned int a=10, b=17, c=5, d=3;
float f;

(1) f = b/c; ()

(2) !(a+b)+c&& b+c/2; ()

(3) (a^b) + (c >> 1|d) ()

(4) f=(a-b>0?c/d:c%d); ()

(5) a=2, b=a* ++b; ()

图 2 判断题与表达式题连续考查定义合法性 类型匹配和运算规则

2.3 整数除法与类型转换

计算顺序：先按运算数原有类型完成运算，再把结果赋给左侧变量。float f=a/c; 中若 a、c 都是 int，先做整数除法，之后才转换为 float。

```
int a = 5, c = 3;

float x = a / c;    /* x 为 1.0 */

float y = (float)a / c; /* y 约为 1.666667 */

float z = 3.0f * c / a; /* z 为 1.8 */
```

强制转换范围：强制转换只作用于紧随其后的表达式。需要改变整个除法时，应转换至少一个除法运算数。

2.4 无符号运算

有符号整数与同等级无符号整数混合时，通常先转换为无符号类型。题目若明确 int 为 16 位，则 unsigned int 的范围为 0 至 65535。

```
unsigned int a = 5, c = 3; /* 假设 16 位 */
unsigned int t = c - a;    /* 65534 */
unsigned int q = (c - a) / 2; /* 32767 */
```

检查方法：出现 unsigned 后，不要直接按负数继续算。先把结果化到 0 至 2^n-1 的范围，再进行后续运算。

2.5 逻辑运算与短路

表达式	确定结果的条件	不会执行的部分
$A \ \&\& \ B$	A 为 0 时结果立即为 0	B
$A \ \ B$	A 非 0 时结果立即为 1	B
$!A$	A 为 0 得 1 否则得 0	无

逻辑运算结果只能是 0 或 1，但左侧已经执行的自增、自减仍然生效。例如 $d-- \ || \ expr$ 在 d 原值非零时不计算 expr，同时 d 已减一。

2.6 自增 自减 条件与逗号运算

- 前置自增先修改再取值，后置自增先取原值再修改。
- 条件运算符只计算被选择的一个分支。
- 逗号运算符从左到右计算，整个表达式的值是最后一个子表达式的值。
- 函数实参之间没有可靠的从左到右求值保证，不能凭书写顺序推断同一变量的多次修改。

标准 C 的边界：若同一完整表达式中对同一标量对象存在未排序的多次修改，或修改与无关读取，例如 $a++/a$ ，这属于未定义行为。旧题若按教材约定给数值，只能作为特定课程约定理解，规范程序中不得这样写。

2.7 位运算

运算	本质	常用判断
$x \& y$	对应位同时为 1 才为 1	取位 清零 掩码
$x y$	对应位有一个为 1 即为 1	置位 合并标志
$x \wedge y$	对应位不同为 1	翻转位 判断差异
$\sim x$	每一位取反	必须结合题目给定的位宽
$x \ll n$	左移 n 位	无溢出时近似乘 2^n
$x \gg n$	右移 n 位	无符号数高位补 0

位运算必须先把整数写成题目指定位宽的二进制。不要把逻辑运算 $\&\&$ 、 $\|\$ 与按位运算 $\&$ 、 $|$ 混用。

2.8 表达式题固定步骤

1. 抄出各变量的声明，标记有符号、无符号、整型和浮点型。
2. 按优先级补上必要括号，先处理短路、条件运算和逗号运算的执行范围。
3. 逐步计算并同步记录自增、自减产生的副作用，不跨步骤心算。
4. 检查整数除法、位宽和最终赋值转换，再写出题目要求的变量值或表达式值。

三 控制结构与程序运行过程

读程序题不是猜输出，而是执行代码。每遇到循环、函数调用或静态变量，都应建立状态记录。

3.1 分支结构的边界

区间必须互斥且完整：连续区间应统一处理端点。例如先判断最高档，再逐级向下判断，能减少重复条件。

```
double rate;

if (amount > 100)    rate = 0.60;

else if (amount > 50) rate = 0.70;

else if (amount > 20) rate = 0.80;

else if (amount > 10) rate = 0.90;

else                rate = 1.00;
```

switch: case 标签必须是整型常量表达式。若不希望继续执行后续 case，应在当前分支末尾写 break。

3.2 循环的四项检查

1. 循环前：参与累计、计数、比较的变量是否已经初始化。
2. 循环条件：最后一个需要处理的元素是否被包含。
3. 循环体：每一轮真正修改了哪些变量。
4. 循环后：输出的是当前值、旧值还是下标转换后的值。

嵌套循环：外层通常控制行、轮次或已确定区域，内层负责当前行、比较区间或剩余元素。排序时内层上界必须随已确定区域缩小。

3.3 break 与 continue

- break 立即结束其所在的最内层循环或 switch。
- continue 跳过本轮剩余语句，直接进入下一轮条件判断或更新表达式。
- 位于 continue 之后且同属当前循环体的普通语句不会执行。

3.4 输出题状态表

轮次	进入时变量	执行分支	输出	离开时变量
1	按初始化填写	写明 if 或 else	精确记录空格与换行	执行自增后填写
2	复制上一轮离开值	继续判断	只写实际执行的输出	继续更新
...	不得跳步	遇 break 标记结束	保留格式	停止

得分		五、输出程序运行结果(Read the program and write the result.) (25分)
评卷人		

(1)

```
#include<stdio.h>
int main()
{
    char str[3][20] = { "basic", "fortran", "c language" };
    int i;
    for (i = 0; i < 3; i++)
    {
        printf("%s\n", *(str+i)+2-i);
    }
    return 0;
}
```

sic
ortran
c language

(2)

```
#include<stdio.h>
void func();

int n;
void func()
{
    int y = 5;
    static int x = 5;
    printf("x=%d,y=%d,n=%d\n", x--, y--, n--);
}
void main()
{
    int i;
    int n = 3, x = 3, y = 3;
    for (i = 0; i < 4; i++)
```

图 3 输出题常把字符串偏移和 static 变量放在同一大题中

3.5 static 与同名变量

- 未显式初始化的全局变量和静态变量自动初始化为 0。
- 普通局部变量每次进入函数都会重新建立；static 局部变量只初始化一次。
- 函数内部的局部变量会遮蔽同名全局变量，但不会改变全局变量本身。
- 不同函数中同名的 static 局部变量是不同对象。

```
void f(void)
{
    int ordinary = 5;
    static int saved = 5;
    printf("%d %d\n", ordinary--, saved--);
}

/* 多次调用时 ordinary 每次从 5 开始，saved 依次减小。 */
```

四 数组 指针与字符串

这一章是整门课的连接点。数组决定数据如何连续存放，指针负责保存地址，字符串是在字符数组上建立的约定。三者必须放在同一套地址模型中理解。

4.1 一维数组与指针

```
int a[5] = {10, 20, 30, 40, 50};
int *p = a;

a[2]    == *(a + 2)
&a[2]   == a + 2
p[2]    == *(p + 2)
```

本质： $a+i$ 是第 i 个元素的地址， $*(a+i)$ 才是元素值。数组名通常转换为首元素指针，但 `sizeof(a)` 和 `&a` 中保留整个数组的含义。

下标范围：长度为 n 的数组合法下标是 0 至 $n-1$ 。可以形成指向尾后位置 $a+n$ 的指针用于比较，但不能解引用。

4.2 二维数组的地址层次

表达式	类型含义	指向内容
<code>a</code>	指向一整行的指针	第 0 行
<code>a+i</code>	指向一整行的指针	第 i 行
<code>*(a+i)</code>	该行首元素指针	第 i 行第 0 个元素
<code>*(a+i)+j</code>	元素指针	第 i 行第 j 个元素
<code>*(*(a+i)+j)</code>	<code>int</code>	<code>a[i][j]</code> 的值

```
#define COLS 4
int matrix[3][COLS];
int (*row_ptr)[COLS] = matrix;
int *element_ptr = matrix[0];
```

关键区别： `int **p` 表示指向 `int*` 的指针；连续二维数组需要的是指向一整行的指针 `int (*p)[COLS]`。两者内存结构不同，不能互换。

4.3 二维数组作为函数参数

```
#define COLS 4

int find_max(int a[][COLS], int rows,
             int *max_row, int *max_col)
{
    int max_value = a[0][0];
    *max_row = 0;
    *max_col = 0;

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < COLS; ++j) {
            if (a[i][j] > max_value) {
                max_value = a[i][j];
                *max_row = i;
                *max_col = j;
            }
        }
    }

    return max_value;
}
```

接口逻辑： 函数返回最大值，行号和列号通过指针参数带回。主函数调用时传入 `&row` 和 `&col`；函数内部通过 `*max_row`、`*max_col` 修改主函数变量。

```
int main()
{
    int m arix[4][4], i, j;
    int m ax, row , colum n;
    for( i=0; i<4; i+ )
        for( j=0; j<4; j+ )
            scanf("%d", &m arix[i][j]); // scanf("%d", &m arix[i][j]); 1 分
    m ax = process(m arix, 4, row , colum n);
    // m ax = process(m arix, 4, & row , & colum n); 1 分
    printf("m ax=%d, lies in row %d and colum n %d\n", m ax, row , colum n);
    return 0;
}

int process( int **p, int n, int prow, int pcolumn n ) // int process( int (*p)[4], int n, int
*prow, int *pcolumn n ) 0.5 分
{
    int i, j;
    int m ax; // int m ax = p[0][0]; 或者 int m ax = -32768; 0.5 分
    for( i=0; i<4; i+ )
        for( j=0; j<4; j+ )
            if( p[i][j] > m ax ) // if( p[i][j] > m ax ) 0.5 分
            {
                m ax = p[i][j];
                prow = i; // *prow = i; 0.5 分
                pcolumn n = j; // *pcolumn n = j; 0.5 分
            }
    return m ax;
}
```

解答内容不得超过装订线

得分		四、程序填空（10分）。
评卷人		(1) 对候选人得票进行统计：设有 4 个候选人，N 个人参加选举，每次输入一个得票候选人的名字，要求最后输出个人的得票结果。

```
#include <stdio.h>
#include <string.h>

#define N 10

struct person
{
    char name[20];
    int count;
};

void main()
```

图 4 二维数组改错题集中检查列宽 指针形参 地址传递和最大值初始化

4.4 字符数组 字符串常量与字符指针

写法	存储形式	能否修改字符	常见用途
char s[]="HUST";	数组中保存字符副本	可以	需要原地修改字符串
const char *p="HUST";	p 指向字符串常量	不可以	只读访问
char s[3][20];	连续的二维字符数组	可以	多个定长字符串
const char *p[3];	三个字符指针组成的数组	取决于所指对象	字符串表和指针排序

字符串成立条件：字符序列末尾必须存在 `\0`。仅用字符列表初始化且没有结尾空字符时，它只是字符数组，不能安全地交给 `%s`、`puts`、`strlen` 等字符串接口。

4.5 输入输出中的地址

```
int number;

char ch;

char word[80];

char line[200];

scanf("%d", &number);

scanf(" %c", &ch);    /* 前导空格跳过先前空白 */

scanf("%79s", word);   /* 数组名已经是首元素地址 */

fgets(line, sizeof line, stdin);
```

- 读普通标量需要传地址，因此 `number` 和 `ch` 前有 `&`。
- 读字符串时数组名已经提供首字符地址，不再写 `&word`。
- `%s` 遇空白停止；读取整行应使用 `fgets`，并处理可能保留的换行符。
- `gets` 无法限制输入长度，已从 C11 标准删除。旧试卷中需要理解其历史行为，但规范代码应使用 `fgets`。

4.6 字符串库函数

函数	结果	必须检查的条件
<code>strlen(s)</code>	不含结尾空字符的长度	<code>s</code> 必须正确结束
<code>strcmp(a,b)</code>	小于 等于或大于 0	不能只假定返回 -1 或 1
<code>strcpy(dst,src)</code>	复制完整字符串	目标空间足够且区域不重叠
<code>strcat(dst,src)</code>	追加到 <code>dst</code> 末尾	<code>dst</code> 容量足够且原本已结束

删除字符时不要重叠 `strcpy`：`strcpy` 的源区和目标区重叠时行为未定义。最稳定的做法是使用读指针和写指针一次完成复制。

```

void delete_digits(char *s)
{
    char *read = s;
    char *write = s;

    while (*read != '\0') {
        if (*read < '0' || *read > '9') {
            *write++ = *read;
        }
        ++read;
    }
    *write = '\0';
}

```

4.7 多级指针的拆解

多级指针题必须从右向左处理“先变化谁，再解引用谁”。先加括号确定自增作用对象，再逐层写出当前指向关系。

1. 画出字符串数组 str：每个元素保存一个字符串首地址。
2. 画出二级表 p：每个元素保存 str 中某个位置的地址。
3. 记录三级指针 pp 当前指向 p 的哪一项。
4. 每遇到 ++pp、++*pp 或 (*pp)++，先完成规定的前置或后置动作，再逐层解引用。
5. 得到 char* 后，末尾的 +n 才表示从字符串第 n 个字符开始输出。

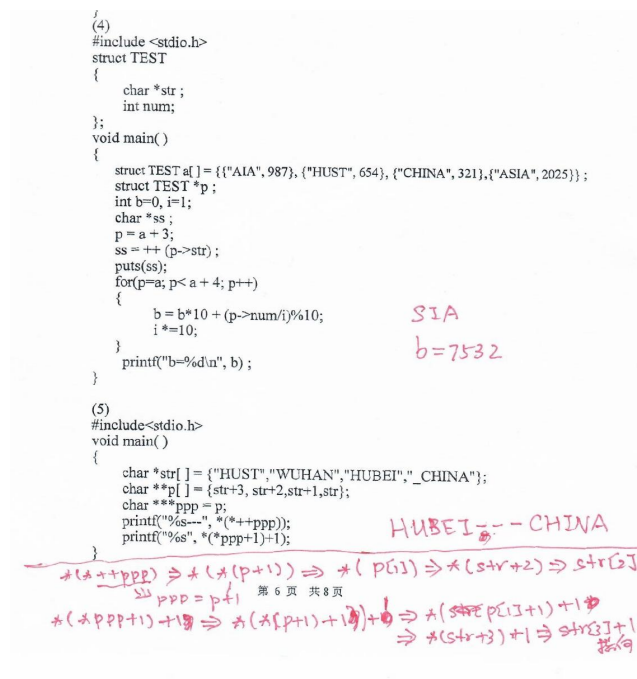


图 5 多级指针和字符串偏移是历年输出题的固定组合

五 函数 存储类别与结构体

5.1 函数接口先于函数体

完整程序应先明确函数接收什么、返回什么、哪些结果需要写回主调函数。接口不清楚时，函数体即使局部正确也无法组成完整程序。

```
#include <stdio.h>

void square_results(int a, int b, int *sum, int *difference);

int main(void)
{
    int x, y, sum, difference;
    scanf("%d%d", &x, &y);
    square_results(x, y, &sum, &difference);
    printf("%d %d\n", sum, difference);
    return 0;
}

void square_results(int a, int b, int *sum, int *difference)
{
    *sum = a * a + b * b;
    *difference = a * a - b * b;
}
```

- C 语言参数传递本质上都是值传递。传入地址后，函数得到的是地址值的副本，但可通过该地址修改原对象。
- 需要返回多个结果时，常用多个指针形参。
- 函数声明、定义和调用三处的返回类型、参数个数与参数类型必须一致。
- 标准入口写法是 `int main(void)` 或 `int main(int argc, char *argv[])`，执行成功返回 0。

5.2 作用域 生存期与存储类别

对象	作用域	生存期	默认初值
普通局部变量	所在代码块	每次进入至离开代码块	不确定值
static 局部变量	所在代码块	整个程序运行期	0
全局变量	从声明处到文件相应范围	整个程序运行期	0
register 局部变量	所在代码块	通常同普通局部变量	不确定值

register: 它只是向实现提出寄存器存储建议，编译器可以忽略。标准 C 不允许对声明为 register 的对象使用取地址运算符。

5.3 递归与动态内存的低频考点

递归: 必须有能直接返回的终止条件，每次递归调用都应使问题规模靠近终止条件。读输出时先写调用栈，再按返回顺序计算。

```
int gcd(int a, int b)
{
    return b == 0 ? a : gcd(b, a % b);
}
```

动态内存: 按所需元素个数申请，立即检查返回值，使用结束后释放。sizeof *p 能随指针目标类型变化，比重复写类型更稳妥。

```
#include <stdlib.h>

int *p = malloc((size_t)n * sizeof *p);
if (p == NULL) {
    return 1;
}

/* 使用 p[0] 至 p[n-1] */

free(p);

p = NULL;
```

5.4 结构体及结构体指针

```
struct Student {
    char name[20];
    char gender;
    float score[4];
};

struct Student students[10];
struct Student *p = students;

students[0].score[0] = 90.0f;
p->gender = 'M';
```

- 普通结构体对象用点运算符访问成员；结构体指针用箭头运算符访问成员。
- p->member 等价于 (*p).member，括号不可省略。
- 结构体可以整体赋值，因此排序时可用一个同类型临时变量交换整条记录。
- 若姓名使用字符数组，结构体赋值会复制数组内容；单独修改姓名仍需使用字符串函数或逐字符处理。

5.5 学生与候选人题的统一数据流

步骤	学生成绩题	候选人计票题
输入	逐人读取姓名 性别 各科成绩	逐票读取候选人姓名
计算	总分 平均分 课程平均分	匹配姓名后 count 加一
排序	按总分或指定课程排序整条记录	通常不需要或按票数排序
筛选	性别 及格线 低于平均分等组合条件	只统计有效候选人
输出	输出符合条件的完整记录	输出姓名和票数

函数划分：输入、计算、排序、查找或筛选、输出分别写函数。main 只负责按正确顺序调用，能显著降低综合题的逻辑混乱。

六 编程大题的固定母题

编程题看似变化很多，实际长期围绕四个母题：数值迭代、数组矩阵、字符串处理和结构体综合。训练时要写完整程序，不能只写核心循环。

分数	
评卷人	

六、编写程序（35 分）注意：不得使用全局变量，注意程序结构

- (1) 编程对所输入的分进行约分，例如，输入 72/192，输出结果为 3/8，若输入 72/24，则输出 3。(7 分)
- (2) 已知 $x_1=1$ ， $x_2=2$ ， $x_n=0.7x_{n-1}+0.3x_{n-2}$ ，求 $\{x_n\}$ 前 10 项的平均值 (8 分)
- (3) 输入一行包含若干单词的字符串，单词之间用空格分开，统计字符串中以 “ing” 结尾的单词个数，要求功能模块设计合理 (10 分)
- (4) 建立一个学生班的成绩登记表，包括的信息有班号（全班一个），总人数（设为 20 人），制表日期（整个表一个）和所有学生的信息。每个学生包含下列信息：学号、姓名、性别、四门课程的成绩。要求：(a) 输入登记表及每个学生的上述所有信息，并统计每个学生的平均成绩；(b) 按平均成绩从高到低的顺序输出排名前 30% 的每个学生的学号、姓名、平均成绩一览表；(c) 统计平均成绩在 60 分以下学生的人数以及其中男女生的比例，并输出统计结果。编写函数完成上述功能要求，主函数中完成功能函数的调用，平均成绩保留两位小数。(10 分)

图 6 历年编程题反复组合数值 字符串和学生信息处理

6.1 数值类 级数求和

以 $\cos(x)$ 为例，直接反复计算幂和阶乘容易重复计算并产生溢出。应由上一项递推得到下一项。

```

#include <math.h>

#include <stdio.h>

int main(void)
{
    double x, sum = 1.0, term = 1.0;
    const double eps = 1e-6;
    int k = 1;

    scanf("%lf", &x);
    do {
        term *= -x * x / ((2.0 * k - 1.0) * (2.0 * k));
        sum += term;
        ++k;
    } while (fabs(term) >= eps);

    printf("%.8f\n", sum);
    return 0;
}

```

- sum 与 term 必须按首项初始化。
- 符号、分子和分母同时包含在递推关系中。
- 精度条件检查的是最后加入的项，因此先产生并累加该项，再判断是否继续。
- 使用 fabs 处理浮点数绝对值。

6.2 数组类 最大值与位置

最大值必须用数组中的合法元素初始化，不能默认设为 0，否则全负数组会得到错误结果。行列位置通过指针参数返回。第四章的 find_max 函数可直接作为标准模板。

排序检查：交换的是数据对象而不是循环下标；结构体排序必须交换整条结构体记录；字符串指针数组排序可交换指针，二维字符数组排序需复制字符内容。

1、求 10 个浮点数的最大值和平均值，其中在 `main()` 函数中完成输入和显示求得的结果，在 `fun` 函数中求最大值和平均值。

```
#include<stdio.h>

void main()
{
    int i;
    float x[10], max, average;

    for(i=0;i<10;i++)
        scanf("%f",&x[i]);
    average=fun(x,10,max);
    printf ("max=%f,average=%f\n", max, average) ;
}

fun(float *p, int n, int *pmax)
{
    float temp;
    int i;

    for(i=0;i<n;i++)
    {
        if (*pmax<p[i])
            *pmax=p[i];
        temp+=p[i];
    }
    return temp/n;
}
```

2、求 5 个字符串中的最小字符串。

```
#include <stdio.h>

void main()
{
    char *pstr[5]={"pascal","basic","prolog","cobol","lisp"};
    char *pmin;
    pmin=getminstr(*pstr,5);
}
```

图 7 数组极值和字符串比较是程序改错中的高频接口题

6.3 字符串类 删除与压缩

删除字符的本质不是反复移动整个后缀，而是让读指针扫描原串，让写指针只保留符合条件的字符。最后补一个结尾空字符。

```

void delete_lowercase(char *s)
{
    char *r = s;
    char *w = s;

    while (*r != '\0') {
        if (*r < 'a' || *r > 'z') {
            *w++ = *r;
        }
        ++r;
    }
    *w = '\0';
}

```

可迁移性：只要更换“保留条件”，同一框架即可完成删除数字、删除小写字母、保留字母或去除特定字符。

6.4 字符串类 提取整数并求最大值

扫描过程中维护“当前是否位于整数内部”。遇到可作为负号的字符或数字时构造当前值；整数结束时更新最大值。这样不需要把所有数字子串先存入二维数组。

```

#include <limits.h>

long max_integer_in_text(const char *s, int *found)
{
    long best = LONG_MIN;
    *found = 0;

    while (*s != '\0') {
        int sign = 1;
        long value = 0;

        if (*s == '-' && s[1] >= '0' && s[1] <= '9') {
            sign = -1;
            ++s;
        } else if (*s < '0' || *s > '9') {
            ++s;
            continue;
        }

        while (*s >= '0' && *s <= '9') {
            value = value * 10 + (*s - '0');
            ++s;
        }

        value *= sign;

        if (!*found || value > best) {
            best = value;
            *found = 1;
        }
    }

    return best;
}

```

- 负号只有在其后紧跟数字时才属于数字子串。
- 字符串可能以数字结束，因此不能依赖“后面一定出现分隔符”才能保存结果。
- 必须区分“最大值为负数”和“没有找到任何整数”。

- 若题目允许超出 long 范围，还需增加溢出检查。

6.5 结构体类 完整组织方式

综合题的稳定写法是先定义结构体和函数原型，再在 main 中按输入、计算、排序、筛选输出的顺序调用。以下为核心框架。

```
#define N 10

#define SUBJECTS 4

#define MATH 0

struct Student {
    char name[20];
    char gender;
    float score[SUBJECTS];
};

void input_students(struct Student s[], int n);
void course_average(const struct Student s[], int n,
                    float avg[SUBJECTS]);
void sort_by_math(struct Student s[], int n);
void print_selected(const struct Student s[], int n,
                    float math_average);

int main(void)
{
    struct Student students[N];
    float average[SUBJECTS];

    input_students(students, N);
    course_average(students, N, average);
    sort_by_math(students, N);
    print_selected(students, N, average[MATH]);
    return 0;
}
```

完整性检查：每个原型必须与定义一致；每次输入都有正确地址和宽度限制；平均值累加器先清零；排序交换整条记录；筛选条件用 && 同时连接；输出字段与题目要求一一对应。

- 1、某门课程最终成绩分别'A'、'B'、'C'、'D'，90 分以上（含 90 分）为'A'，70-89 分为'B'，60-69 分为'C'，60 分以下为'D'。现要求从键盘输入成绩的等级，在屏幕上输出该成绩等级对应的成绩范围（8 分）
- 2、编写一个函数求一个二维数组列累加和的最大值并返回列号；另编写 main 测试函数完成二维数组数的输入，函数调用，输出列累加和的最大值以及列号（9 分）。
- 3、输入一行包含若干单词（假定不超过 10 个）的字符串，单词之间用空格分开，要求删除该行字符串中最大的那个单词（假定最大单词只有一个）形成新的字符串，并显示。（不考虑单词的合法性以及语句的语法）（9 分）
- 4、编程处理某班 30 个学生 4 门课的成绩，它们分别是数学、物理、英语和计算机，按编号从小到大的顺序依次输入学生的姓名、性别和四门课的成绩，计算每个学生的平均成绩，并以清晰的格式从低分到高分的顺序打印平均分低于 80 分的学生成绩（9 分）

图 8 最新综合题仍以成绩计算 排序 条件筛选和分函数实现为主

七 考前一页易错检查表

这张表用于每次写完程序后的最后检查。检查应按编译、类型、逻辑、边界和输出五层进行，不能只看代码是否“像答案”。

层次	必须检查的问题	正确标准
编译	头文件 原型 返回类型 括号 分号	声明 定义 调用完全一致
类型	格式符 指针层级 二维数组列宽	实参与形参 指针与目标对象匹配
初始化	累计值 计数器 最大值 静态变量	最大值来自合法元素 累计值有明确初值
循环	初值 条件 更新 break continue	恰好处理全部目标元素且不越界

层次	必须检查的问题	正确标准
字符串	容量 输入方式 结尾空字符	目标空间足够 最终一定写入 \0
函数	值传递与地址传递	需要写回的结果使用指针或返回值
结构体	成员访问和交换对象	点与箭头正确 排序交换整条记录
输出	格式符 空格 换行 字段顺序	与题目要求逐项对应

最后必须能独立完成的五类程序

- 带边界的分段计算或简单计算器。
- 使用递推关系并按精度停止的级数求和。
- 二维数组极值 排序 鞍点或图形输出。
- 字符串删除 统计 单词处理或数字子串提取。
- 结构体数组的输入 计算 排序 查找 筛选和输出。

如果以上五类仍有一类只能“看懂”而不能完整写出，就应继续针对该类真题进行独立重写。